

EECE 7205: Fundamentals of Computer Engineering

Fall 2026

Lab Assignment #1:

Student Name	Hritvik Kuppa
NUID	003947137
Email	kuppa.h@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	24 Sept 2026

Submission Checklist (must match syllabus requirements)

- Submit **one PDF** generated from this Overleaf report.
- Include **clear screenshots** for each task (text must be readable).
- Include your **GitHub profile link** and **Overleaf project link** in the report.
- File name: `Assignment01_LastName.pdf`

How to complete this assignment

1. Complete each task below in order.
 2. For each task, include the required evidence (screenshots, links, and short notes).
 3. When you paste a link, ensure it opens correctly (no private/local-only links unless instructed).
-

Problem 1

1. Understand the Problem

- **Goal:** Given a set of intervals, sort them by their starting value and merge overlapping intervals.
- **Input:**
 - The number of intervals.
 - Each interval has a start and end value.
- **Output:** Sorted intervals with no overlaps.
- **Edge cases:**
 - No intervals overlap.
 - All intervals overlap.

2. Design the Solution

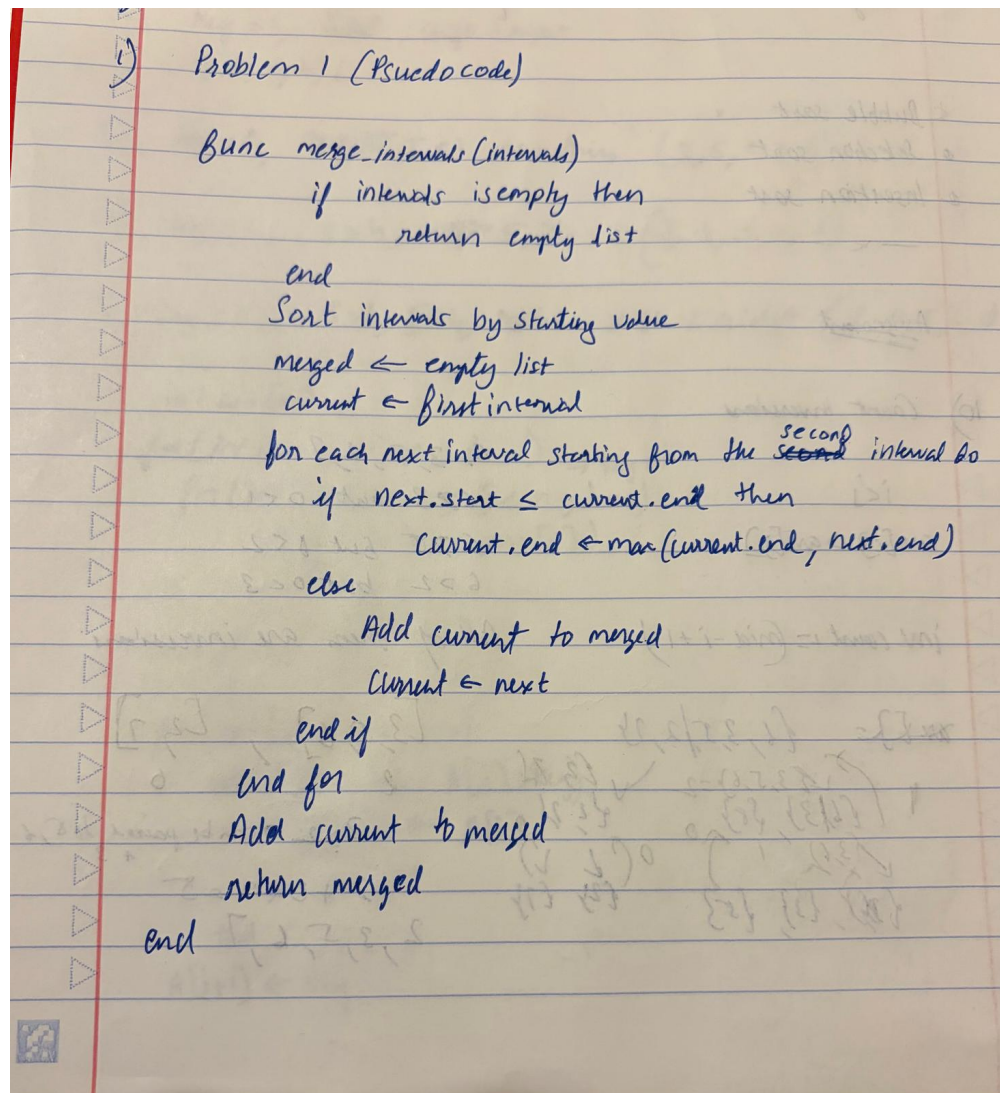
- **Algorithm:**

1. Sort the intervals by their starting values using Bubble Sort.
2. Check each interval with the next interval for an overlap.
3. If an overlap exists, merge the intervals by updating the end value.
4. If there is no overlap, keep the current interval and move to the next one.

- **Data Structure:**

A 2D array is used because each interval contains two values: a start and an end. Each row represents one interval.

3. Write Pseudocode



```
1) Problem 1 (Pseudocode)

func merge_intervals(intervals)
    if intervals is empty then
        return empty list
    end
    Sort intervals by starting value
    merged ← empty list
    current ← first interval
    for each next interval starting from the second second interval do
        if next.start ≤ current.end then
            current.end ← max(current.end, next.end)
        else
            Add current to merged
            current ← next
        end if
    end for
    Add current to merged
    return merged
end
```

Figure 1: Pseudocode for Problem 1

4. Explain Correctness

The algorithm first sorts the intervals by their starting values. This allows the end value of the current interval to be compared with the starting value of the next interval. If the next interval starts before or at the end of the current interval, the two intervals overlap and can be merged.

If there is no overlap, the current interval is added to the result and the next interval becomes the current interval. Therefore, all overlapping intervals are merged and the resulting intervals do not overlap.

5. Analyze Complexity

- Check if intervals are empty: 1
- Sort using Bubble Sort: n^2
- Set merged to empty: 1
- Set current to first interval: 1
- Check each next interval: $n - 1$
- Check overlap condition: $n - 1$
- Merge overlapping intervals: $n - 1$
- Add current interval to merged: n

$$T(n) = 1 + n^2 + 1 + 1 + (n - 1) + (n - 1) + (n - 1) + n$$

$$T(n) = n^2 + 4n$$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int in;
int temp; //temp variable for sorting

cout << "Enter number of intervals: ";

//Asking the user to enter the number of intervals since arrays are static
//and have a fixed size, so I am asking to enter the number of intervals
cin >> in;

int interval[in][2];

//Entering the intervals
for (int i=0; i<in; i++){
    cout << "Enter the start and end of interval " << i+1 << ": ";
```

```

    //The i+1 is to basically show the interval number they are entering,
    //like interval 1, interval 2 etc..
    cin >> interval[i][0] >> interval[i][1];
}

//Sorting - Bubble sort with a temp variable
for (int i=0; i<in-1; i++){
    for (int j=0; j<in-1; j++){
        if (interval[j][0] > interval[j+1][0]){

            //checking if the value in the previous interval is greater or not
            temp = interval[j][0];
            interval[j][0] = interval[j+1][0];
            interval[j+1][0] = temp;

            //This is for the second value in the interval
            temp = interval[j][1];
            interval[j][1] = interval[j+1][1];
            interval[j+1][1] = temp;
        }
    }
}

//To check for overlaps and merge them
int merged = 0;

for(int i=1; i<in; i++){

    //If the current merged interval overlaps with the next interval,
    //then it keeps the larger ending value
    if (interval[merged][1] >= interval[i][0]){
        interval[merged][1] =
            max(interval[merged][1], interval[i][1]);
    }
    else{
        merged++; //moving to the next position

        //copying the current interval into next merged position
        interval[merged][0] = interval[i][0];
        interval[merged][1] = interval[i][1];
    }
}

//To print out the intervals
for (int i=0; i<=merged; i++){
    cout << "[" << interval[i][0] << "," << interval[i][1] << "];"

    if (i != merged) {

```

```

        cout << ", ";
    }
}

```

7. Test the Solution

Normal case:

```

PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE"
Choose a problem from 1-10: 1
Enter number of intervals: 5
Enter the start and end of interval 1: 1 3
Enter the start and end of interval 2: 2 6
Enter the start and end of interval 3: 8 10
Enter the start and end of interval 4: 9 12
Enter the start and end of interval 5: 15 18
[1,6],[8,12],[15,18]
PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 2: Normal case test for Problem 1

Edge case:

```

PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE"
Choose a problem from 1-10: 1
Enter number of intervals: 4
Enter the start and end of interval 1: 1 3
Enter the start and end of interval 2: 2 5
Enter the start and end of interval 3: 4 7
Enter the start and end of interval 4: 6 10
[1,10]
PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 3: Edge case test for Problem 1

Problem 2

1. Understand the Problem

- **Goal:** Sort an array containing only 0s, 1s, and 2s so that all 0s come first, followed by all 1s, and then all 2s.
- **Input:**
 - An integer n representing the number of elements.
 - An array containing only 0s, 1s, and 2s.
- **Output:** A sorted array containing all 0s first, then all 1s, and then all 2s.
- **Edge cases:**

- Already sorted array.
- Reverse sorted array.
- Empty array.

2. Design the Solution

- **Algorithm:**

Two methods are used: Counting Based and Dutch National Flag (DNF).

- **Counting Based:**

1. Count the number of 0s, 1s, and 2s in the array.
2. Rewrite the array using the counts, placing all 0s first, then 1s, and then 2s.

- **DNF Algorithm:**

1. Use three variables: low, mid, and high.
2. Values before low are 0s.
3. Values between low and mid are 1s.
4. Values between mid and high are unknown.
5. Values after high are 2s.
6. Move the elements into their correct regions until the unknown region is empty.

- **Data Structure:**

An array is used to store the 0s, 1s, and 2s.

3. Write Pseudocode

Counting Based

Problem 2

Counting

func sort-by-count (arr)

count-0 $\leftarrow$ 0

count-1 $\leftarrow$ 1

count-2 $\leftarrow$ 2

for each element in arr do

if element = 0 then

count-0 $\leftarrow$ count-0 + 1

if element = 1 then

count-1 $\leftarrow$ count-1 + 1

else

count-2 $\leftarrow$ count-2 + 1

end if

end for

index $\leftarrow$ 0

for i $\leftarrow$ 1 to count-0 do

arr[index] $\leftarrow$ 0

index $\leftarrow$ index + 1

end for

Figure 4: Counting-based pseudocode

```

for i ← 1 to count0 do
    arr[index] ← 0
    index ← index + 1
end for

for i ← 1 to count1 do
    arr[index] ← 1
    index ← index + 1
end for

return arr
end function

```

DNF Based

B) DNF Algo

```

func DNF(arr)
    low ← 0
    mid ← 0
    high ← n-1
    while mid ≤ high do
        if arr[mid] = 0 then
            swap(arr[low], arr[mid])
            low ← low + 1
            mid ← mid + 1
        else
            mid ← mid + 1
        end if
    end while
end func

```

Figure 5: DNF-based pseudocode

4. Explain Correctness

Counting:

After the input is taken, the number of 0s, 1s, and 2s are counted in the array. The array is then rewritten using the counts. Therefore, all 0s are placed first, followed by all 1s and then all 2s.

```

else if arr[mid] = 1 then
    mid ++
else
    swap(arr[mid], arr[high])
    high --
end if
end while
end func

```

For example, if there are three 0s, three 1s, and two 2s, the resulting array is:

$[0, 0, 0, 1, 1, 1, 2, 2]$

DNF:

The DNF algorithm maintains three regions:

- 0 to $low - 1$ consists of zeros.
- low to $mid - 1$ consists of ones.
- mid to $high$ contains unknown elements.
- $high + 1$ to $n - 1$ consists of twos.

If the middle element is 0, it is moved to the zero region. If it is 1, it is already in the correct region. If it is 2, it is moved to the two region. This continues until all elements are placed in their correct regions.

5. Analyze Complexity

Counting:

- Initializing counters: 1

- Checking each element: n
- Incrementing a counter: n
- Initializing index: 1
- Placing 0s: n
- Placing 1s: n
- Placing 2s: n
- Return array: 1

$$T(n) = 5n + 2$$

$$T(n) = O(n)$$

DNF:

- Initializing low, mid, high: 1
- Checking the while condition: $n + 1$
- Checking the current value: n
- Swap/increment/decrement: n
- Return array: 1

$$T(n) = 3n + 3$$

$$T(n) = O(n)$$

6. Implement in C++

```
int choice;
int n;

cout << "Enter the number of elements: ";
cin >> n;

vector<int> arr(n);

for (int i=0; i<n; i++){
    cout << "Enter element " << i+1 << ": ";
    cin >> arr[i];
}

cout << "Enter the number to select the method "
```

```

        << "(1 = Counting based, 2 = One pass DNF Algorithm): ";
cin >> choice;

//A menu to choose either Counting based or DNF
switch(choice){

    case 1: {

        //Counting based
        int count_0 = 0;
        int count_1 = 0;
        int count_2 = 0;

        //Checking for 0,1,2 in the array
        //The count of 0 increases whenever it finds 0 in the array
        //and similar for 1,2
        for (int i=0; i<n; i++){
            if(arr[i] == 0)
                count_0++;
            else if(arr[i] == 1)
                count_1++;
            else
                count_2++;
        }

        //Replacing the elements based on the count of 0s, 1s and 2s.
        int index = 0;

        for (int i=0; i<count_0; i++){
            arr[index++] = 0;
        }

        for (int i=0; i<count_1; i++){
            arr[index++] = 1;
        }

        for (int i=0; i<count_2; i++){
            arr[index++] = 2;
        }

        for (int i=0; i<n; i++){
            cout << arr[i] << ",";
        }

        break;
    }

    case 2: {

```

```

//One Pass DNF

//Initializing mid, high and low values.
int mid = 0;
int high = n-1;
int low = 0;
int temp;

while(mid <= high){

    if (arr[mid] == 0){

        //If the middle value is 0, swap the value with the low
        //and increment mid and low
        temp = arr[low];
        arr[low] = arr[mid];
        arr[mid] = temp;

        mid++;
        low++;
    }

    //If arr[mid] is 1, no need to swap anything
    //since that's where 1 should be
    else if (arr[mid] == 1)
        mid++;

    else{

        //When value = 2, swap with high and decrement high
        temp = arr[high];
        arr[high] = arr[mid];
        arr[mid] = temp;

        high--;
    }
}

//Printing the array
for (int i=0; i<n; i++){
    cout << arr[i] << ",";
}

break;
}

```

7. Test the Solution

Normal case (Counting):

```
Choose a problem from 1-10: 2
Enter the number of elements: 6
Enter element 1: 2
Enter element 2: 0
Enter element 3: 2
Enter element 4: 1
Enter element 5: 1
Enter element 6: 0
Enter the number to select the method (1 = Counting based, 2 = One pass DNF Algorithm): 1
0,0,1,1,2,2,
```

Figure 6: Normal case using the Counting method

Normal case (DNF):

```
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE\" ; if ($?) { g++ Assignment
Choose a problem from 1-10: 2
Enter the number of elements: 6
Enter element 1: 2
Enter element 2: 0
Enter element 3: 1
Enter element 4: 2
Enter element 5: 1
Enter element 6: 1
Enter the number to select the method (1 = Counting based, 2 = One pass DNF Algorithm): 2
0,1,1,1,2,2,
PS C:\Users\hritv\Desktop\NEU\FOCE>
```

Figure 7: Normal case using the DNF method